

Reinforcement Learning

Part 6: a language model is an agent

Part 6 of 7. Text as an MDP, reward models, RLHF, and DPO.

You have been training a policy this whole time

A language model produces a distribution over the next token given everything so far, and you sample from it:

$$a \sim \pi_{\theta}(\cdot \mid s)$$

That is not an analogy. That is the definition of a stochastic policy from lecture 1, and a transformer satisfies it exactly.

So every tool from the last four lectures applies. The only genuinely new question is: **what is the reward?**

For chess it is winning. For Atari it is the score. For “write a helpful, honest answer” nobody can write the function down – and that single missing piece is what this lecture is about.

Outline

Text generation as an MDP

Where the reward comes from

Reinforcement learning from human feedback

Direct preference optimisation

The same five symbols

$(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, read for text.

The translation

Symbol	In the gridworld	In a language model
$\mathcal{S}$	which square you are on	the prompt plus every token generated so far
$\mathcal{A}$	up, down, left, right	every token in the vocabulary ($\sim 50\text{k} - 150\text{k}$)
$P(s' s, a)$	you might slip	deterministic : append the token, that is the new state
$R(s, a)$	-0.04 per move	zero , until the response ends
γ	0.9	usually 1 – a token now is worth a token later
$\pi(a s)$	an arrow per square	the softmax over the vocabulary

One episode = one response. It starts at the prompt, each action appends a token, and it ends at the end-of-sequence token – at which point a single number arrives saying how good the whole thing was.

What is strange about this MDP

Property	Consequence
Deterministic transitions	No environment stochasticity at all. Every bit of randomness comes from the policy's own sampling. Half the machinery for handling P is unnecessary.
Enormous action space	$\arg \max_a Q(s, a)$ over 100k actions at every step, with a Q value to learn for each. Value-based methods are out – this is why the LLM world is policy-based.
Reward only at the end	Maximal credit-assignment difficulty. A 2,000-token answer gets one number, and every token must somehow be assigned blame.
The policy already works	Unlike a robot flailing at random, a pretrained model starts out fluent. You are <i>adjusting</i> a good policy, not discovering one – so tiny, careful steps.
No simulator to reset	You cannot re-run the same conversation with a different action and compare. Every rollout is a fresh sample.

Row four is the one that shapes everything. A pretrained model is an enormously valuable starting policy, and RL here is fine-tuning it without wrecking it. **Everything in this lecture is a mechanism for not moving too far.**

Nobody can write *R*

So learn it from the only people who can judge.

Ask for comparisons, not scores

You cannot write “helpfulness” as a function. You could ask humans to *rate* answers 1–10, and it goes badly: one rater’s 7 is another’s 4, and everyone drifts over a long session.

Comparisons are much easier

“Which of these two answers is better?” is a question people answer consistently, quickly, and with far better inter-rater agreement than any absolute scale.

InstructGPT collected rankings of **4–9 outputs** per prompt and turned each ranking into all its pairwise comparisons.

So the data is a set of triples (x, y_w, y_l) : a prompt, a preferred answer, and a rejected one.

Now turn preferences into a number.

Bradley-Terry: from comparisons to a reward

Assume every answer has a hidden quality $r(x, y)$, and that the chance a human prefers y_w grows with the gap. The Bradley-Terry model (1952) says:

$$\begin{aligned} P(y_w \succ y_l \mid x) &= \frac{\exp r(x, y_w)}{\exp r(x, y_w) + \exp r(x, y_l)} && \text{a softmax over two options} \\ &= \sigma(r(x, y_w) - r(x, y_l)) && \text{divide through by } \exp r(x, y_w) \end{aligned}$$

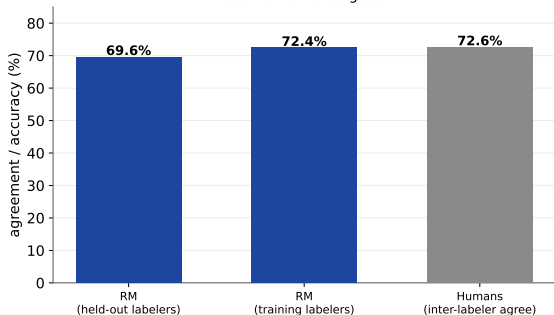
Fit r_ϕ by maximum likelihood on the observed preferences, and the loss is just logistic regression on the *difference* of two scores:

$$\mathcal{L}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right]$$

Note r_ϕ is only ever **pinned down up to a constant** – adding c to every score changes no difference. Remember that; DPO uses it.

The reward model is a model, with all that implies

The reward model predicts a preference about as well as two humans agree



It is usually the pretrained model with the language head swapped for a single scalar output. InstructGPT used a **6B** reward model to train a 175B policy.

And it is fitted on finite data. It agrees with humans on held-out comparisons somewhat better than humans agree with each other – which is a low bar, and it is wrong in ways nobody has catalogued.

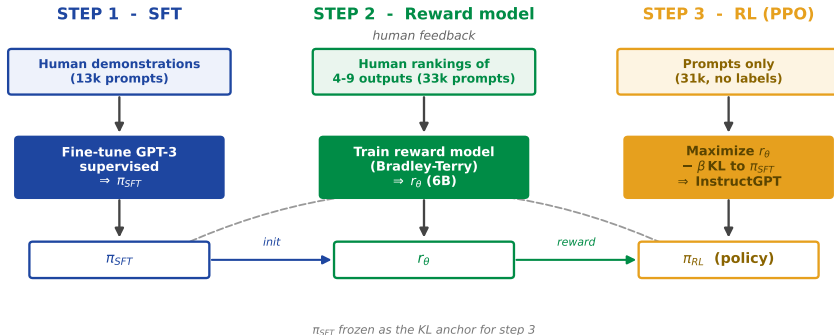
You are about to optimise hard against this thing. Lecture 5's reward-hacking section was preparation for exactly this moment: the policy will search for the places where the reward model is wrong, because those are where the reward is highest.

Reinforcement learning from human feedback

RLHF: demonstrate, then compare, then optimise.

The pipeline

RLHF: supervised demos $\rightarrow$ a learned reward $\rightarrow$ RL against it



Step 1 is behaviour cloning – lecture 5’s imitation learning, on text. Step 2 learns the reward nobody could write. Step 3 is the only part that is reinforcement learning, and it is PPO from lecture 4 with one extra term.

The objective, and the term that is not in lecture 4

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} \left[\underbrace{r_{\phi}(x, y)}_{\text{learned reward}} \right] - \beta \underbrace{\text{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))}_{\text{stay near the SFT model}}$$

The KL term is not a regulariser in the chapter-2 sense. It is a **leash**, and it is doing three jobs at once:

- ▶ **Keeps the reward model in-distribution.** r_{ϕ} was trained on samples from roughly π_{ref} . Wander far and its scores become meaningless.
- ▶ **Preserves what pretraining bought.** Fluency, knowledge and grammar are not in the reward signal, so nothing else protects them.
- ▶ **Is the trust region.** Lecture 4's whole point, appearing here in explicit form rather than as a clip.

In practice PPO's clip *and* this KL penalty are both used. Belt and braces.

What happens without the leash

Set $\beta = 0$ and the model finds the exploit

With no KL term the policy is free to go anywhere that scores well under r_ϕ . What it reliably finds is not better answers but **adversarial input to the reward model**: degenerate, repetitive text that happens to sit in a corner where r_ϕ is high.

Reward goes up. Quality collapses. The two curves separate, and only one of them is on your dashboard.

So β is the most important number in RLHF. Too small and the model hacks the reward. Too large and nothing changes – you have paid for an RL pipeline to reproduce the SFT model.

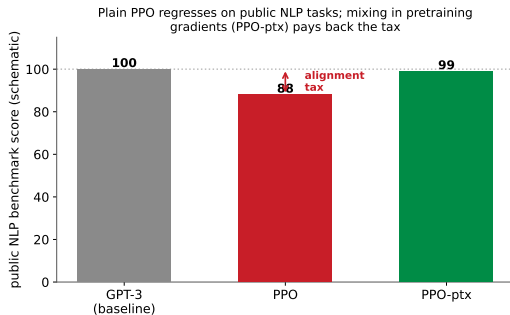
Practitioners watch the KL, not the reward. *Reward going up while KL runs away is the signature of a failed run*, which is the same lesson as lecture 4's entropy diagnostic.

Did it work?

The InstructGPT headline

Labelers preferred the **1.3B** InstructGPT model's outputs to those of the **175B** GPT-3.

A 100× smaller model, preferred – because the bigger one was never trained to do what was asked, only to continue text.



The alignment tax

Aligned models got *worse* on some standard NLP benchmarks. The fix InstructGPT used was to mix pretraining gradients back into the RL updates (PPO-ptx).

Worth naming plainly: **optimising for what users want costs you something on what benchmarks measure.** That trade-off has not gone away.

The bill

Look at what has to be in memory during step 3:

Model	Role	Trained?
π_θ	the policy being improved	yes
π_{ref}	frozen SFT model, for the KL term	no
r_ϕ	the reward model	no
V_ψ	the critic, for advantages (lecture 4)	yes

Four models, two of them being trained. The critic is typically the same size as the policy, so a 70B RLHF run holds something like $70\text{B} \times 4$ parameters plus optimiser state. And then generation: every update needs fresh samples from the current policy, so training is interleaved with inference. RLHF is an infrastructure problem as much as a learning one.

Two obvious questions follow. *Do we need the RL loop?* (this lecture) and *do we need the critic?* (next lecture)

Delete the reward model and the RL loop

One of the neater derivations in modern machine learning.

Step 1: the objective has a closed-form solution

Forget algorithms for a moment and just *solve* the RLHF objective. For a fixed prompt x , maximising reward minus β KL over all possible distributions has a known answer:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

- ▶ Take the reference model and **re-weight** it: answers the reward likes get multiplied up, answers it dislikes get multiplied down.
- ▶ β controls the sharpness. Large β : barely move. Small β : pile all the mass on the highest-reward answer.
- ▶ $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(x, y)/\beta)$ normalises it – and it is **hopeless to compute**, being a sum over every possible response.

So we know exactly what we want, and cannot build it. That $Z(x)$ is the only obstacle – and the next slide makes it disappear.

Step 2: run the equation backwards

Take logs and solve for the reward instead of the policy:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Any policy implicitly defines a reward. So make the policy the reward model, and substitute this into Bradley-Terry:

$$\begin{aligned} P(y_w \succ y_l) &= \sigma(r(x, y_w) - r(x, y_l)) \\ &= \sigma\left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} + \cancel{\beta \log Z(x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \cancel{\beta \log Z(x)}\right) \end{aligned}$$

$Z(x)$ **cancels** – it depends only on the prompt, and Bradley-Terry only ever sees *differences*. Same trick as the dynamics cancelling in lecture 2.

The DPO loss

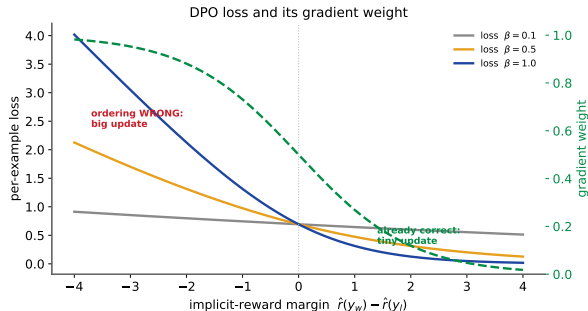
$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Read what is *not* there:

RLHF needs	DPO needs
a trained reward model	nothing – the policy is its own implicit reward
sampling from the policy during training	no generation at all
a critic network	none
PPO, clipping, advantages, GAE	a single cross-entropy-shaped loss
four models in memory	two, one of them frozen

It trains like supervised fine-tuning. That is the entire appeal, and it is why **Direct Preference Optimization (DPO)** took over open-source alignment almost immediately (Rafailov et al., 2023).

What the loss actually does



The gradient weight is largest exactly where the model has the ordering **wrong**, and vanishes once a pair is comfortably correct.

It is logistic regression on the implicit-reward margin – so it inherits the behaviour you already know from chapter 3.

β plays the same role as in RLHF: how far the policy may drift from π_{ref} .

A subtlety worth knowing: nothing forces $\pi_{\theta}(y_w|x)$ to go *up*. The loss only cares about the *margin*, so it can be reduced by pushing the rejected answer down hard while the preferred one also drifts down. In practice both probabilities often fall – which is not what anyone intends

What you give up

DPO

Simple, stable, cheap. No generation, no critic, no reward model.

But it is offline. It only ever sees the fixed pairs in your dataset – answers produced by *some other* model.

PPO / online RL

Expensive and fiddly.

But it is online. It generates fresh answers from the current policy and scores those, so it can discover behaviours that appear nowhere in the preference data.

This is lecture 5's offline-vs-online distinction, exactly. DPO is offline RL on text, and it inherits the ceiling: *a policy trained on a fixed dataset is bounded by what that dataset contains.*

You cannot DPO your way to a reasoning ability that is not already present in some answer somebody preferred. That limitation is why the next lecture exists.

The family that followed

Identity Preference Optimisation (IPO), **Kahneman-Tversky Optimisation (KTO)**, **Odds Ratio Preference Optimisation (ORPO)** and **Simple Preference Optimisation (SimPO)**. Each removes a different requirement from DPO:

Method	The complaint	The change
IPO (Azar et al., 2024)	With clean preferences, DPO's objective can be driven to overfit – it keeps pushing the margin forever.	Replaces the logistic loss with a bounded one, so there is a finite optimum.
KTO (Ethayarajh et al., 2024)	Paired comparisons are expensive to collect.	Learns from unpaired thumbs-up / thumbs-down signals, using a loss shaped by prospect theory. Production systems have this data for free.
ORPO (Hong et al., 2024)	Why two training stages?	Merges SFT and alignment into one objective with an odds-ratio penalty. No reference model, no separate SFT run.
SimPO (Meng et al., 2024)	Keeping a frozen reference model in memory is pure overhead.	Uses the <i>length-normalised average log-probability</i> as the implicit reward. No reference model at all; reported +6.4 on AlpacaEval 2 over DPO.

Which one, in practice

Situation	Reach for
You have preference pairs and limited compute	DPO . It is the default for a reason.
You have thumbs-up/down from a product	KTO . The data you already have is the data it wants.
You want one stage instead of two	ORPO .
You need behaviour that is not in any dataset (reasoning, tool use, code that must run)	Online RL – and that is the next lecture.

Do not over-read the leaderboard deltas. These methods are close enough that the result usually depends more on the preference data than on which loss you picked. The honest summary is that *data quality dominates the choice of objective* – which is the LIMA lesson from the seminar track.

Recap

- ▶ A language model **is** a stochastic policy. State = tokens so far, action = next token, episode = one response, reward = one number at the end.
- ▶ The action space rules out value-based methods, and the fact that the policy already works means every method here is a mechanism for **not moving too far**.
- ▶ Humans compare better than they score. **Bradley-Terry** turns comparisons into a reward model – and that model is only defined up to a constant.
- ▶ **RLHF** = SFT, then a reward model, then PPO with a **KL leash** to the reference policy. Without the leash the policy hacks the reward model instead of improving. Watch KL, not reward.
- ▶ It costs **four models** in memory and interleaves training with generation.
- ▶ **DPO** solves the RLHF objective in closed form, inverts it so the policy is its own reward model, and the intractable $Z(x)$ cancels inside Bradley-Terry. What is left trains like supervised learning.
- ▶ DPO is **offline**, so it is bounded by its dataset. IPO, KTO, ORPO and SimPO each remove one more requirement.

Next: what if the reward needs no human at all – just a checker that says whether the answer is right? RLVR, GRPO, DeepSeek-R1, and the failure modes the field spent 2025 fixing.